

Poarta care spune nu

Exerciții · `compose-gate.yaml` · nginx ca strat de apărare

Până acum nginx a fost un poștaş: a dus cererea mai departe. Aici învață și să **refuze** — pe criterii de identitate, de mărime, de timp și de debit.

Fiecare pas produce intenționat un cod de eroare. `401`, `413`, `429`, `504` nu sunt accidente aici, sunt criteriul de gata: dacă nu-l poți provoca, nu l-ai configurat.

`api` e un serviciu care face la comandă orice îi ceri: `/get` răspunde imediat, `/delay/9` după nouă secunde, `/bytes/2000000` trimite doi megaocteți. E ținta de test pentru tot ce urmează.

Nu primești `gate.conf`, certificatul și nici fișierul de parole. Toate trei se fac înainte de primul `up`.

Pasul 1 — Cheile, înainte de orice

Compose-ul montează trei lucruri care încă nu există. Dacă pornești fără ele, Docker creează directoare goale cu numele lor și nginx moare — verifică asta o dată, ca să știi cum arată.

```
openssl req -x509 -newkey rsa:2048 -nodes -days 365 \
  -subj "/CN=IP_PUBLIC" \
  -keyout certs/lab.key -out certs/lab.crt

docker run --rm httpd:2.4-alpine \
  htpasswd -nbB sebi PAROLA_TA > htpasswd
```

CRITERIU DE GATA

```
ls -l certs/ htpasswd
docker compose -f compose-gate.yaml ps
```

Patru containere *running*. Fișierul `htpasswd` trebuie să fie fișier, nu director. Parola nu se comite niciodată — nici hash-ul.

Pasul 2 — Totul pe HTTPS, și nimic pe HTTP

Portul 443 servește platforma, portul 80 nu mai servește nimic: doar trimite clientul mai departe. În plus, nginx nu trebuie să-și mai anunțe versiunea, și adaugi două antete de securitate.

CRITERIU DE GATA

```
curl -s -o /dev/null -w '%{http_code} %{redirect_url}\n' http://IP/api/get
curl -sk -o /dev/null -w '%{http_code}\n' https://IP/api/get
curl -skI https://IP/ | grep -iE 'server|x-'
```

`301` către adresa `https`, apoi `200`. La `Server:` trebuie să scrie doar `nginx`, fără număr de versiune. Explică în predare de ce `-k` e necesar și ce anume verifică el, de fapt.

Pasul 3 — O zonă cu parolă

`/files/` e interfața de fișiere. Nu trebuie să ajungă la ea nimeni care nu are parola din pasul 1. Restul platformei rămâne deschisă.

CRITERIU DE GATA — trei coduri

```
C='curl -sk -o /dev/null -w %{http_code}\n'
$C https://IP/files/
$C -u sebi:PAROLA_TA https://IP/files/
$C -u sebi:gresit https://IP/files/
```

401 , 200 , 401 . Dacă al doilea nu e 200 , problema e în fișierul de parole, nu în configurație.

Pasul 4 — Cât de mare are voie să fie o cerere

Pe /api/ limita e 1 MB. Pe /files/ , unde oamenii chiar încarcă fișiere, 50 MB. Deci limita nu e una singură pentru tot serverul.

```
dd if=/dev/zero of=big.bin bs=1M count=3
curl -sk -o /dev/null -w '%{http_code}\n' -X POST \
  --data-binary @big.bin https://IP/api/post
```

CRITERIU DE GATA

413 pe /api/ . Apoi găsește singur cât de mare trebuie să fie fișierul ca să treacă, și scrie numărul. Explică de ce răspunsul vine de la nginx și aplicația nu află niciodată de cererea asta.

Pasul 5 — Cât de mult are voie să aștepte

O aplicație care se gândește nouă secunde blochează un worker de nginx nouă secunde. Pune-i o limită de cinci.

CRITERIU DE GATA — două coduri și două durate

```
C='curl -sk -o /dev/null -w %{http_code}_%{time_total}\n'
$C https://IP/api/delay/2
$C https://IP/api/delay/9
```

Primul 200 , al doilea 504 după aproximativ cinci secunde — nu după nouă. Scrie ambele durate.

Pasul 6 — Cât de des are voie să ceară

Limitezi /api/ la 5 cereri pe secundă de client, cu voie să depășească scurt. Apoi lovești poarta cu 50 de cereri deodată:

```
for i in $(seq 50); do
  curl -sk -o /dev/null -w '%{http_code}\n' https://IP/api/get &
done | sort | uniq -c
```

CRITERIU DE GATA

Două numere: câte au întors 200 și câte 429 . Suma e 50. Apoi explică de ce numărul de 200 -uri nu e nici 5, nici 50, ci exact cât ți-a ieșit — răspunsul e în al doilea parametru pe care l-ai scris, nu în primul.

Pasul 7 — Să nu mai întrebe de două ori

Ruta /lent/ duce la același serviciu, dar prin memorie: primul care cere plătește așteptarea, ceilalți nu. Pune /lent/delay/2 în cache și trimite un antet care spune, la fiecare răspuns, dacă a venit din memorie sau nu.

CRITERIU DE GATA — trei cereri identice

```
for i in 1 2 3; do
  curl -sk -D h.txt -o /dev/null -w '%{http_code} %{time_total}\n' \
    https://IP/lent/delay/2
  grep -i x-cache h.txt
done
```

Prima: aproximativ două secunde, cu starea **MISS**. Următoarele două: sub o zecime de secundă, cu **HIT**. Pune cele trei durate în predare.

Ce predai

- `gate.conf`, comis în `lectii/L2-server-nginx/poarta-restrictiva/`
- toate codurile și duratele cerute la pașii 2–7, într-o listă
- răspunsurile în scris la provocări

Nu comiți: `certs/`, `htpasswd`, `big.bin`. Verifică `git status` înainte, nu după.

Când ceva nu merge

Ce vezi	Unde te uiți întâi
proxy-ul nu pornește, în log <code>is a directory</code>	ai pornit înainte să existe certificatul sau fișierul de parole. Pasul 1
<code>curl</code> se plânge de certificat	corect, e semnat de tine. <code>-k</code> . Provocarea 1
<code>401</code> și cu parola bună	fișierul de parole: format, cale, sau montat ca director
bucula de la pasul 6 dă numai <code>200</code>	cererile n-au plecat deodată. Fără <code>&</code> nu testezi nimic
la pasul 7 vezi <code>MISS</code> de fiecare dată	cheia diferă între cereri, sau răspunsul n-are voie în cache
<code>504</code> unde nu te aștepți	limita de la pasul 5 se aplică peste tot unde ai pus-o, nu doar pe ruta lentă

Provocări

1. Certificatul tău face browserul să urle. Ce ar trebui ca să nu mai urle, cine dă lucrul ăla, și de ce nu-l poți obține pentru adresa pe care o folosești acum? Trei fraze, plus ce ai schimba concret ca să devină posibil.

2. Limita de la pasul 6 numără pe adresa clientului. Dacă mâine pui platforma în spatele altui proxy — un load balancer, Cloudflare — toți clienții ajung la tine cu aceeași adresă. Ce se strică, cum îți dai seama, și ce folosești în loc?

3. Pune o rută cu parolă în cache, cere-o cu utilizatorul tău, apoi cere-o fără parolă. Ce ai scurs? Arată dovada și spune ce anume din cheia de cache ar fi trebuit să fie altfel.

4. Verifică antetele de securitate de la pasul 2 pe *fiecare* rută, nu doar pe rădăcină. Pe cel puțin una lipsesc. Nu e o scăpare de-a ta — e o regulă de moștenire din nginx. Găsește-o, explic-o într-o frază și repar-o.